Rapport de tests « utilisateur vicieux »

TocGameOnline — 08/10/2026 — branche `nardis_dev`

Comment j'ai testé

- ~40 comptes créés via l'API (emails `@test.fr` / `@t.fr`), tout passe par nginx (`https://localhost:8443`)
- Scripts HTTP : entrées pourries, doubles clics, requêtes en parallèle, tokens volés ou périmés
- Clients socket.io : vraie partie 2 humains + 2 bots, déco/reco au hasard, actions bidon, spam du chat
- Chromium sans tête : clics au hasard partout, F5, bouton retour, hors-ligne, 2 onglets, largeur mobile

● bloquant / risque éliminatoire ● un correcteur le verra ● finition
REPRO reproduit en vrai, pas supposé

Bloquant / sécurité

1 « Create game » / « Join » depuis le menu : lobby bloqué sur « Connecting... » REPRO CORRIGÉ

- **Corrigé :** `autoConnect: false` enlevé de `chatSocket.ts`. Vérifié dans le navigateur : lobby sans F5 (hôte et invité), chat global sans doublon.
- Il faut faire F5 pour voir la salle d'attente. C'est le parcours principal du jeu.
- Cause : `frontend/src/sockets/chatSocket.ts` crée la socket avec `{ autoConnect: false }`. Dans socket.io-client, cette option est retenue par le *Manager*, partagé par toutes les sockets du même serveur. Le chat global est créé en premier (sur /menu), donc la socket du jeu (`gameSocket.ts`) créée ensuite ne se connecte jamais (`socket.js:127 : if (this.io._autoConnect)`).
- En ouvrant /lobby directement (F5), la socket du jeu est créée en premier et tout marche.
- **Fix :** appeler `socket.connect()` juste après `io(...)` dans `gameSocket.ts` (ou enlever `autoConnect: false` du chat).

2 Plus de 4 joueurs dans une partie, sièges en double REPRO CORRIGÉ

- **Corrigé :** le joueur est ajouté en mémoire avant l'`await` Prisma. Testé : 6 joins simultanés → 3 acceptés, 3 refusés, sièges `{0,1,2,3}`.
- 5 personnes cliquent « Join » en même temps sur une table où il reste 3 places : 5 × 201.
- En base, la partie 6 a 6 joueurs, sièges `{0,1,1,1,2,2}`.
- Cause : `game.service.ts joinGame()` vérifie les places libres puis fait un `await prisma…` *avant* d'ajouter le joueur en mémoire, et calcule le siège depuis l'ancien état.
- **Fix :** ajouter le joueur en mémoire (`commit`) avant l'await, ou un verrou par partie.

3 Les cookies de session de tout le monde sont lisibles sans mot de passe REPRO

- Les logs du backend (pino-http) contiennent les en-têtes complets : cookie `jwt` **et** `refresh_token`.
- Ils partent dans Elasticsearch, ouvert sur `http://localhost:9200` sans authentification (`xpack.security.enabled=false`). `curl 'localhost:9200/_search?q=refresh_token'` renvoie des refresh tokens valides 7 jours : on peut se connecter sur le compte de n'importe qui.
- Kibana (5601) aussi est ouvert sans login.
- **Fix :** `redact: ['req.headers.cookie', 'res.headers["set-cookie"]']` dans `backend/src/config/logger.config.ts`, et ne pas publier 9200 / 5601 (ou activer la sécurité).

4 Secrets dans git

- `.env` (racine) est commité, avec `POSTGRES_PASSWORD` (dans l'historique depuis le commit `38339ed`).
- Pas de `.gitignore` à la racine : 118 fichiers de `frontend/.next` sont commités aussi.
- En correction, des identifiants dans le repo sont souvent éliminatoires.
- **Fix :** `.gitignore` (.env, .next, node_modules, uploads), `git rm --cached`, et changer les mots de passe (ils restent dans l'historique).
- Au passage : `.env` contient `POSTGRES_PASSWORD` et `POSTGRES_DB` en double.

5 Erreurs 500 sur des cas banals REPRO

- Inscription avec un email déjà pris : 500 (au lieu de 409). Pareil avec un pseudo déjà pris, et au 2e clic d'un double clic sur « Sign up ».
  Fix : gérer l'erreur Prisma `P2002` dans `auth.service.ts register()` (`profile.service.ts` le fait déjà, on peut copier).
- `GET /api/profile/99999999999999` : 500 (nombre trop grand pour un Int Postgres).
- Compte supprimé mais jwt encore valide (15 min) → `POST /api/game` : 500 (clé étrangère).
- Socket `action` avec un payload `null` : crash « Cannot read properties of null (reading 'gameId') » (`game.gateway.ts handleAction` : `body.gameId` sans `?.`).

Un correcteur le verra

6 Partie à 1 pion par joueur PRIS EN CHARGE

- **En cours :** Nardis s'en occupe (enlever `QUICK_GAME=1` avant la correction).
- `backend/.env` contient `QUICK_GAME=1` : une partie finit en ~40 coups. Pratique pour tester, à enlever avant la correction.

7 Plusieurs parties à la fois / partie en cours perdue REPRO CORRIGÉ

- **Corrigé :** une seule partie à la fois : `create` / `join` répondent 409 si on est déjà dans une partie pas finie (double clic compris). Le menu affiche « Resume game » / « Quit game » à la place de « Create game » / « Join game » quand on a une partie en cours (`GET /api/game/current`, `POST /api/game/leave`). Quitter en pleine partie : l'IA prend la place **pour de bon**, même si le joueur se reconnecte ; plus aucun humain → partie terminée. Testé : double create → 1 partie + 409, quitter en partie → l'autre joueur continue, dernier humain parti → `finished` en base.
- « Create game » pendant une partie : une nouvelle partie est créée et /play montre maintenant celle-là. L'ancienne continue avec l'IA à notre place, sans moyen d'y revenir.
- Double clic sur « Create game » : 2 parties créées. Le menu ne propose pas de « reprendre ma partie ».
- **Fix :** refuser create/join (409) si `findGameOfUser()` renvoie une partie pas finie, ou rediriger vers elle.

8 Pas de bouton « Quitter » dans le lobby CORRIGÉ

- **Corrigé :** bouton « Quit game » dans le lobby (et dans le menu). Le siège est libéré, les sièges sont renumérotés et le joueur suivant devient l'hôte ; la salle reste active tant qu'il y a quelqu'un. Testé : l'hôte part → l'invité passe au siège 0 (mémoire et base) et peut lancer.
- **Reste :** la suppression de compte de l'hôte ne passe pas par « Quit » ; à brancher sur `leaveGame()` si on veut couvrir ce cas.
- Un joueur qui a rejoint garde son siège pour toujours, même s'il part au menu.
- Si l'hôte supprime son compte, le lobby reste là et personne ne peut le lancer (« Only the host can lock the team »).

9 Avatar entre 1 et 2 Mo refusé par nginx REPRO

- Le front et le back disent « max 2 MB », mais nginx garde sa limite par défaut de 1 Mo : 413 avec une page HTML nginx, et le front affiche juste « Something went wrong. ».
- **Fix :** `client_max_body_size 2m;` dans `nginx/nginx.conf`.

10 Le front tourne en `next dev` dans Docker

- Console pleine de « [HMR] Invalid message ... » sur chaque page. Pour la correction : build de prod (`next build && next start`).

11 Erreurs rouges dans la console sur des cas normaux

- « Failed to load resource: 404 » quand on ajoute un ami qui n'existe pas, avec un mauvais code d'invitation, sur /players/999999 ; et 3 × 401 juste après « Log out ».
- Certains correcteurs comptent toute erreur console. À voir : répondre 200 + `{ error }` pour les erreurs « attendues », ou au moins ne plus appeler l'API après le logout.

12 Pseudo et email : la casse n'est pas prise en compte REPRO

- On peut créer « BOB » alors que « bob » existe (ou renommer son profil pareil) : usurpation facile dans le chat.
- Même email en majuscules = 2e compte.
- **Fix :** stocker l'email en minuscules ; comparer les pseudos sans la casse (ou `citext`).

13 Pas de limite de tentatives

- 30 mauvais mots de passe d'affilée : toujours 401, aucun blocage (brute force).
- Chat global : 200 messages envoyés en 41 ms, tous acceptés (flood). Pareil pour le chat de partie.

14 Refresh token et plusieurs onglets REPRO

- 3 refresh en parallèle avec le même token : 201, 201, 401. Deux ont réussi avec le même token (vérification puis mise à jour pas atomiques), et le cookie qui reste n'est plus valide au refresh suivant.
- En vrai : 2 onglets ouverts au moment où le jwt expire → déconnexion « au hasard » (`refreshAuth()` ne mutualise que dans un seul onglet).

15 Ports encore ouverts

- Le backend répond toujours en direct sur 3001 : `docker-compose.yml` est modifié mais les conteneurs n'ont pas été recréés (`docker compose up -d --force-recreate backend frontend nginx`).
- 9200 (Elasticsearch) et 5601 (Kibana) publiés, voir le point 3.

Finitions

16	/login et /register restent accessibles une fois connecté (pas de redirection vers /menu).
17	Après l'inscription on atterrit sur /login au lieu d'être connecté directement.
18	Code d'invitation avec des espaces autour (copier-coller) : « Invalid invite code » (pas de trim). En minuscules ça marche.
19	Hors-ligne 9 s en pleine partie : rien ne l'indique à l'écran (pas de « Reconnexion... »).
20	Le jwt reste valide 15 min après « Log out » (normal avec des jwt, mais à savoir expliquer).
21	La socket des amis envoie 2 fois la connexion au namespace en dev (`40/friends,40/friends,`) : même cause que le chat en double, `connectFriendsSocket()` teste `!socket.connected` au lieu de `!socket.active` .
22	Bio à `null` → message « Bio must be at most 280 characters » (faux message). 200 emojis refusés car la longueur compte en UTF-16 (le front fait pareil, c'est cohérent).
23	Mot de passe de 8 espaces accepté à l'inscription.
24	Un fichier qui commence par l'en-tête PNG puis du HTML est accepté comme avatar. Servi en image/png avec nosniff, donc pas exploitable, mais un vrai décodage d'image serait plus propre.
25	Fichiers parasites dans le repo : `backend/src/game/sedNQSCqD` , `frontend/CLAUDE.MD` .
26	`GET /api` (sans slash) : redirection 301 de nginx vers /api/.
27	Le code d'invitation n'est unique qu'en mémoire : après un redémarrage, un code déjà en base peut être retiré au sort → 500 à la création (très rare).

Ce qui tient bien (rien à faire)

- Pages privées sans être connecté → /login, partout.
- Injection SQL / NoSQL dans le login, jwt forgé « alg none », vieux refresh token rejoué : refusés.
- XSS : la bio et le chat stockent le HTML, mais React l'échappe (aucun `dangerouslySetInnerHTML`).
- Avatar : SVG, faux PNG en texte et path traversal refusés, `nosniff` présent.
- Amis : soi-même, inconnu, doublon, demandes croisées, double accept : tous gérés proprement.
- Partie : start par un non-hôte, double start, join après le start, niveau d'IA bidon, coups illégaux et hors tour : refusés.
- Déco/reco aléatoires pendant 75 s : la partie ne s'est jamais bloquée, l'IA reprend la main après 5 s et le joueur la récupère à son retour ; F5 en pleine partie ramène bien sur /play.
- Tous les humains partis : la partie est terminée proprement.
- Aucun crash JavaScript côté front pendant ~300 clics au hasard.

Nettoyage : les comptes de test (ids 5 à 49, emails `@test.fr` / `@t.fr`) sont restés en base, plus 2 comptes `lva…` / `lvb…@test.fr` créés pour tester le point 7. Le chat global contient ~200 messages « spam N » de test (en mémoire : un restart du backend le vide).